

SYSTEMIQ

**AN AI-POWERED SYSTEM MONITORING AND
OPTIMIZATION PLATFORM**

A PROJECT REPORT PRESENTED BY

L.M.A.Lakshani

(BAD-IT-2324-F-016)

in partial fulfillment of the requirement for the award of the
HIGHER NATIONAL DIPLOMA IN INFORMATION TECHNOLOGY

of the

Badulla ATI

SRI LANKA

2026

DECLARATION

I hereby declare that the work reported in this project report was carried out by me under the supervision of Mr. I.P.J.M. Karunarathna It describes the results of my own independent work, except where due reference has been made in the text. No part of this report has been submitted earlier or concurrently for the same or any other degree.

Date:2026.07.14..... Signature of the Candidate:anushika.....

Certified by:

Supervisor (Name): Mr. I.P.J.M. Karunarathna

Date: Signature:

ABSTRACT

Every computer keeps producing information about how it is running, but most people never get to use it. The tools that come with an operating system, such as Task Manager or the top command, only show what is happening at the current moment. They do not keep a record of the past and they cannot say anything about the future. Because of this, a problem like high memory usage, an overheating processor, or a disk that is almost full is usually noticed only after it has already slowed the user down or caused a failure.

SystemIQ was built to change that. It is a web-based platform that keeps watching the computer's resources, saves the readings so they can be studied over time, and uses machine learning to estimate what the usage will look like in the near future. On top of this it produces plain-language recommendations, raises alerts when something goes wrong, and lets more than one user log in with different levels of access. In this way the system does not just report what is happening; it also helps the user understand why it matters and what is likely to happen next, so that action can be taken before a problem becomes serious.

The system was developed using Python and the FastAPI framework on the server side, React on the browser side, MySQL as the database, and scikit-learn for the machine-learning part. Live data is sent to the browser using WebSocket connections.

Title of the project: SystemIQ — An AI-Powered System Monitoring and Optimization Platform

Name of the Author: ...L.M.A.Lakshani.....

Address of the Author:No-46/2, Kahagolla, Bandarawela.....

ACKNOWLEDGEMENTS

First, I would like to thank my supervisor, Mr. I.P.J.M. Karunaratna , for the guidance, patience, and encouragement given to me throughout this project. The advice I received helped me a great deal whenever I was stuck.

I am also grateful to all the lecturers and staff of Advanced Technological Institute for the knowledge and support they shared during my studies. Finally, I want to thank my family and my friends, who kept encouraging me from the beginning to the end of this work. Without their support, this project would not have been completed.

Thanks

L.M.A.Lakshani

TABLE OF CONTENTS

•	DECLARATION	ii
•	ABSTRACT	iii
•	ACKNOWLEDGEMENTS	iv
•	TABLE OF CONTENTS	v
•	LIST OF FIGURES	vii
•	LIST OF TABLES	viii
•	LIST OF ABBREVIATIONS	ix
•	CHAPTER 1: INTRODUCTION	1
○	1.1 Problem Statement	2
○	1.2 Functional Requirements	2
•	CHAPTER 2: LITERATURE REVIEW AND STUDY OF SIMILAR APPLICATIONS	3
○	2.1 Literature Review	3
○	2.2 Study of Similar Applications	4
○	2.3 Motivation	4
•	CHAPTER 3: MATERIALS AND METHODS	5
○	3.1 Materials	5
▪	3.1.1 Hardware Requirements	5
▪	3.1.2 Software Requirements	5
○	3.2 Methods	6
•	CHAPTER 4: TECHNOLOGIES	8
○	4.1 Frontend and Backend Programming	8
○	4.2 Data Modeling and Management	9
○	4.3 IDEs and Tools	9
○	4.4 Data Model (ER Diagram)	10
○	4.5 Bird's-Eye View of the System	11
○	4.6 UML Diagrams	12
○	4.7 Time Plan (Gantt Chart)	14
○	4.8 Schedule and Budget Summary	15
•	CHAPTER 5: RESULTS AND DISCUSSION	16
○	5.1 Objectives	16
○	5.2 Results	16
○	5.3 Limitations	17
○	5.4 Future Development	17
•	CHAPTER 6: CONCLUSIONS	18
•	FUTURE WORK AND RECOMMENDATIONS	19
•	REFERENCES	20

- APPENDIX A: ADDITIONAL SCREENSHOTS 21
- APPENDIX B: SPECIAL ALGORITHMS 22

LIST OF FIGURES

Figure	Description	Page
Figure 1	SDLC development methodology	6
Figure 2	System architecture	9
Figure 3	ER Diagram	10
Figure 4	Bird's-Eye View of the system	11
Figure 5	Use Case Diagram	12
Figure 6	Class Diagram	13
Figure 7	Gantt Chart	14
Figure 8	Login page	21
Figure 9	Real-time monitoring dashboard	21
Figure 10	User management	21
Figure 11	Predictions page	21
Figure 12	Hardware page	21

LIST OF TABLES

Table	Description	Page
Table 1	Functional requirements	2
Table 2	Hardware requirements	5
Table 3	Software requirements	5
Table 4	Use case descriptions	12
Table 5	Project schedule	15
Table 6	Budget summary	15

LIST OF ABBREVIATIONS

Abbreviation	Meaning
API	Application Programming Interface
CPU	Central Processing Unit
CRUD	Create, Read, Update, Delete
ER	Entity Relationship
HND	Higher National Diploma
JWT	JSON Web Token
ML	Machine Learning
ORM	Object Relational Mapping
RBAC	Role-Based Access Control
REST	Representational State Transfer
SDLC	Software Development Life Cycle
SPA	Single Page Application
UI	User Interface
UML	Unified Modeling Language

CHAPTER 1: INTRODUCTION

Computers and servers are used everywhere today, and keeping them running well has become very important. The problem is that the tools most people use to check the health of a machine only show a single moment in time. They do not save any history, they do not tell the user why something is happening, and they cannot warn about what is coming. So when a computer becomes slow, gets too hot, or runs out of storage, the user normally finds out only after the trouble has already started.

SystemIQ was developed to solve this problem. It is a web-based application that keeps monitoring the resources of a machine in real time and stores the data so that it can be studied later. Using this stored data, the system trains machine-learning models that predict how the resources will be used in the near future. It also creates simple recommendations that any user can understand, sends alerts when limits are crossed, and allows several users to log in with different roles.

The application was built with a Python FastAPI backend, a React frontend, a MySQL database, and scikit-learn for the prediction models. Live readings are pushed to the browser through WebSocket connections, so the dashboard updates on its own without the user having to refresh the page.

1.1 Problem Statement

The main problems that this project tries to solve are:

- Ordinary monitoring tools show only the current value of a resource. There is no record of the past, so trends cannot be studied.
- The user gets no early warning before the memory fills up, the disk runs out, or the hardware overheats. The problem is seen only after it has already happened.
- These tools do not explain what the numbers mean, and they do not suggest what the user should do.
- There is no single and easy-to-use system that brings monitoring, history, and prediction together for one machine and allows more than one user.

1.2 Functional Requirements

The main functions that SystemIQ must perform are listed in Table 2.

Table 1: Functional requirements.

#	Functional Requirement
FR1	Collect and show live CPU, memory, disk, and network readings
FR2	Send the readings to the browser in real time
FR3	Store the readings and show them as charts over a time range
FR4	List the running processes and allow a process to be stopped
FR5	Create recommendations and raise alerts when a limit is crossed
FR6	Predict future resource usage using machine learning
FR7	Let users log in and control access according to their role
FR8	Analyze files and logs and explain them in simple language
FR9	Read hardware sensors such as temperature, fan speed, and battery
FR10	Answer questions about the system in natural language

CHAPTER 2: LITERATURE REVIEW AND STUDY OF SIMILAR APPLICATIONS

2.1 Literature Review

2.1.1 How operating systems report resource usage

Every operating system keeps track of how its resources are being used. On Linux this information is available through special files such as those under `/proc` and `/sys`, which hold details about the processor, memory, and hardware sensors. Common tools like `top`, `htop`, and `vmstat` simply read this information and print it on the screen. The weakness of these tools is that they only show the value at that moment and do not keep it. SystemIQ reads the same information (through the `psutil` library, which works on many operating systems) but saves it, so the data can be looked at over a period of time.

2.1.2 Working with time-based data and predicting it

When a value such as CPU usage is recorded again and again at regular times, it forms what is called a time series. Predicting the next values of a time series is a topic that has been studied for a long time. The methods range from simple ones like averages and linear regression to more advanced machine-learning methods such as decision-tree groups. For predicting only a short time ahead, using the recent past values as inputs usually works well. SystemIQ uses a Random Forest model with recent values as inputs to predict CPU and memory usage, and a linear model for slower changes such as disk growth.

2.1.3 Real-time dashboards on the web

To show data that keeps changing inside a web browser, the server needs a way to push the new values to the browser by itself. The old method of asking the server again and again is slow and wastes resources. The WebSocket protocol solves this by keeping one open connection through which the server can send updates at any time. When this is combined with a modern framework such as React, which updates only the part of the page that changed, the result is a smooth live dashboard. SystemIQ uses this approach and refreshes the readings about every two seconds.

2.1.4 Moving from reacting to predicting

In recent years, the way systems are monitored has been changing. Instead of only reacting after a problem happens, newer approaches try to predict problems before they occur by studying past data with machine learning. This idea is often called AIOps. SystemIQ follows the same idea, but on a small and self-hosted scale, by joining together prediction, risk levels, and easy-to-read recommendations.

2.2 Study of Similar Applications

A few existing applications were studied to understand what is already available and where the gaps are.

Application	Strengths	Weakness that SystemIQ overcomes
top / htop	Light and fast, real time	No history, no prediction, works only in the terminal
Task Manager / GNOME System Monitor	Simple graphical view	No prediction, no recommendations, single user
Grafana with Prometheus	Powerful charts	Hard to set up, no built-in prediction or advice
Datadog / New Relic	Very complete	Paid, cloud based, and expensive

Compared to these, SystemIQ brings prediction, plain-language questions, and safe optimization together in one light and self-hosted package that runs on a single machine.

2.3 Motivation

The free tools stop at showing the current value, while the professional tools are heavy and cost money. There was a clear need for a light system that a student or a small office could host by itself and that still adds some intelligence, such as history, prediction, and advice, on top of ordinary monitoring. This gap is what motivated me to build SystemIQ, because it can save time when finding the cause of a problem and can even help stop the problem before it happens.

CHAPTER 3: MATERIALS AND METHODS

3.1 Materials

3.1.1 Hardware Requirements

Table 2: Hardware requirements.

Component	Server / Host	Client PC
Processor	Four cores or more	Dual core or better
Memory (RAM)	4 GB minimum (8 GB recommended)	4 GB
Disk space	About 10 GB free (more for long history)	Standard
Operating system	Linux (e.g. Ubuntu) for full sensor support	Any OS with a modern browser

3.1.2 Software Requirements

Table 3: Software requirements.

Part	Software used
Backend	Python 3.12, FastAPI, Uvicorn
Database	MySQL 8, SQLAlchemy, Alembic
Machine learning	scikit-learn, pandas, NumPy
System readings	psutil
Frontend	Node.js, Vite, React 18, Tailwind CSS, Recharts, Axios
Browser	Chrome, Firefox, or Edge

3.2 Methods

For this project I followed the iterative and incremental method of the Software Development Life Cycle. Instead of trying to build the whole system at once, I built one feature at a time. Each feature was designed, coded, and tested before I moved on to the next one. The order I followed was monitoring first, then history and analytics,

then recommendations and alerts, then the machine-learning part, then login and user roles, and finally the hardware sensor part. Working this way made it easier to test each part on its own and to fix problems early.

The main stages of the work were:

1. Studying the requirements and deciding what the system had to do.
2. Designing the structure of the system and the database.
3. Building each feature step by step.
4. Testing the features, using automated tests written with pytest.
5. Improving the appearance and speed, and writing the documentation.

Git was used for version control during the whole project so that the history of the code was kept and changes could be rolled back if needed. Figure 2 shows this development method.

Software Development Lifecycle Process

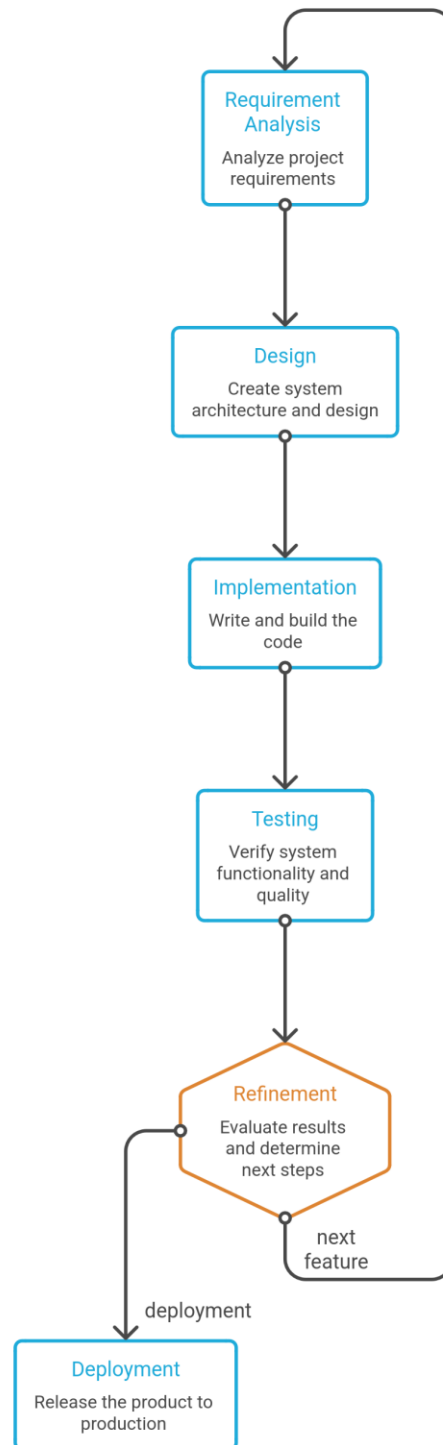


Figure 1: SDLC development methodology.

CHAPTER 4: TECHNOLOGIES

4.1 Frontend and Backend Programming

The frontend, which is the part the user sees, was built as a Single Page Application using React 18 with the Vite build tool. Tailwind CSS was used for the styling, Recharts for the charts, and Axios for sending requests to the server. The frontend talks to the backend in two ways: through normal REST requests for actions, and through a WebSocket for receiving the live readings.

The backend was built with FastAPI on Python 3.12. It is organized in three layers so that the code stays clean and easy to change:

1. The interface layer holds the FastAPI routes and the WebSocket. It deals only with receiving requests, checking who the user is, and sending back the answers.
2. The service layer holds all the real logic of the system. There is one module for each area, such as monitoring, analytics, recommendations, prediction, alerts, login, and hardware.
3. The data layer holds the database models and the connection to the database.

The backend also runs a few tasks in the background. One task collects and sends the readings about every two seconds, another saves a reading and checks for alerts and recommendations about every minute, and a third retrains the prediction models once a day.

4.2 Data Modeling and Management

The data is described using the SQLAlchemy ORM and is stored in a MySQL database. The structure of the database is kept up to date with Alembic migrations, so the tables always match the code. Figure 3 shows the overall structure of the system.

System Architecture Flowchart

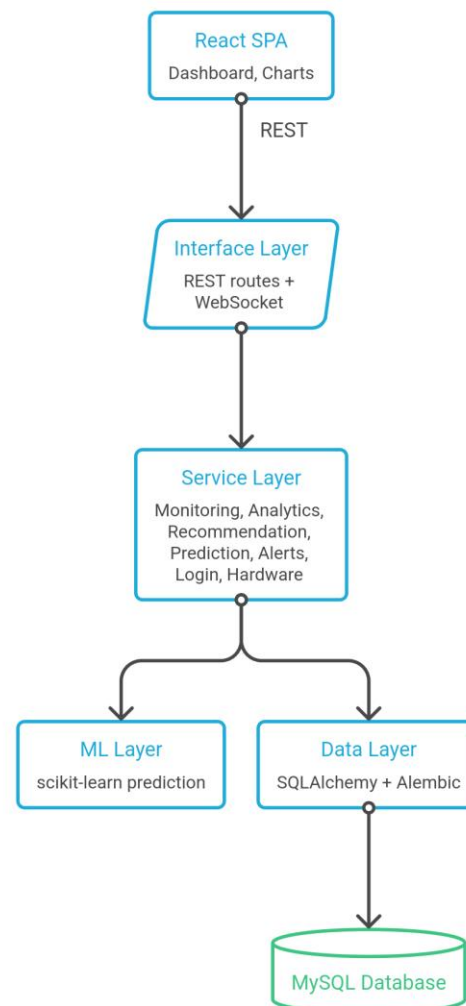


Figure 2: System architecture.

4.3 IDEs and Tools

The tools that were used during development are:

- Visual Studio Code as the main code editor
- Git for version control
- MySQL Workbench and the MySQL command line for managing the database
- The built-in Swagger UI page for testing the API

4.4 Data Model (ER Diagram)

The main tables of the database and their most important columns are shown below. The ER diagram in Figure 4 shows the same tables.

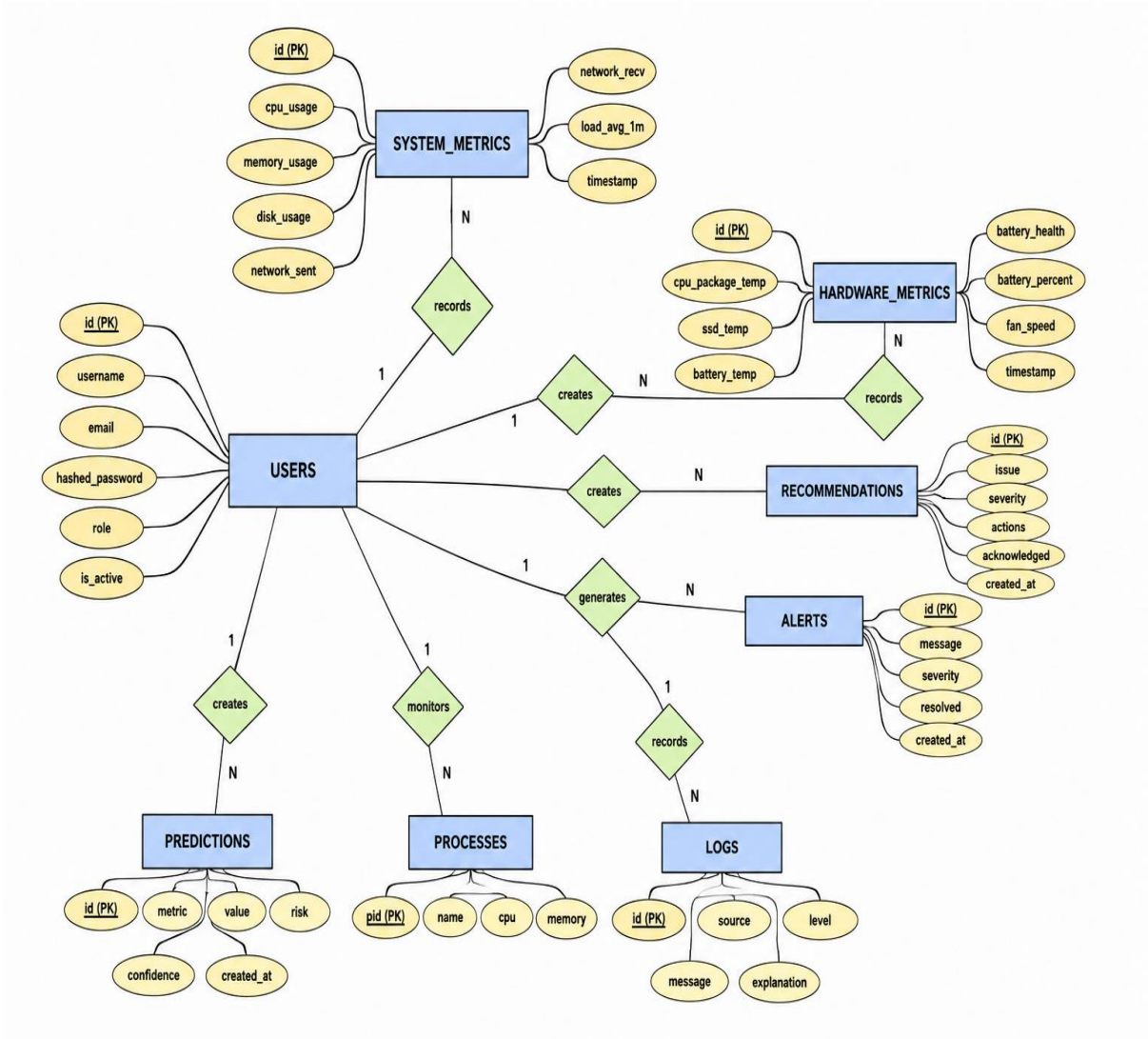


Figure 3: ER Diagram.

Table	Purpose	Main Columns
users	User accounts	id, username, email, hashed_password, role, is_active
system_metrics	Saved resource readings	id, cpu_usage, memory_usage, disk_usage, network_sent, network_recv, load_avg_1m, timestamp
hardware_metrics	Sensor readings	cpu_package_temp, ssd_temp, battery_temp, battery_health, battery_percent, fan_speed
recommendations	Advice created by the system	id, issue, severity, actions, acknowledged, created_at
alerts	Warnings when a limit is crossed	id, message, severity, resolved, created_at
predictions	Saved prediction results	id, metric, value, risk, confidence, created_at
processes	Snapshots of running processes	pid, name, cpu, memory
logs	Analyzed log lines	id, source, level, message, explanation

4.5 Bird's-Eye View of the System

Figure 5 gives a simple overall picture of how the parts of the system work together.

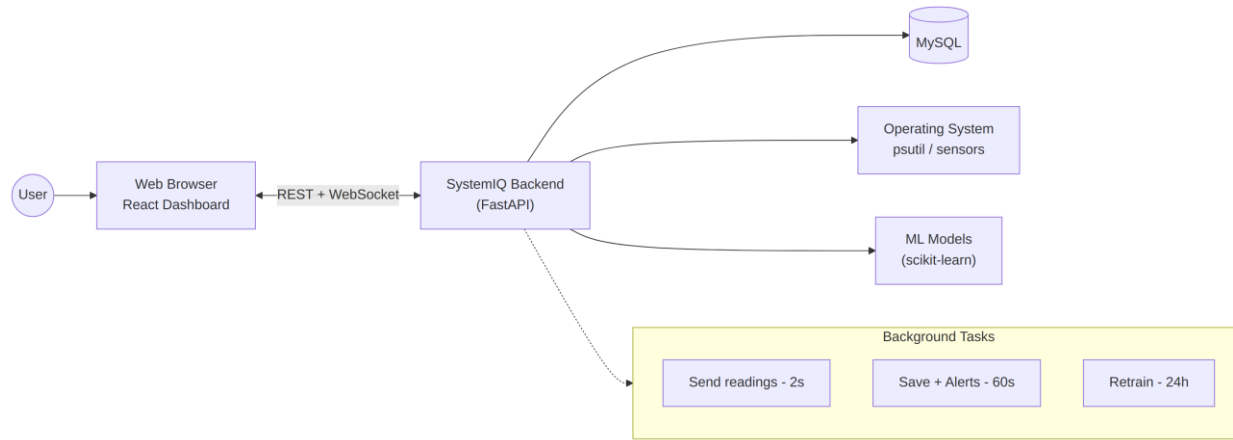


Figure 4: *Bird's-Eye View of the system.*

4.6 UML Diagrams

4.6.1 Use Case Diagram

The system has two kinds of users, the Administrator and the ordinary User. The Administrator can do everything the User can do, and in addition can manage the user accounts. Figure 6 shows the use case diagram, and Table 5 explains each use case.

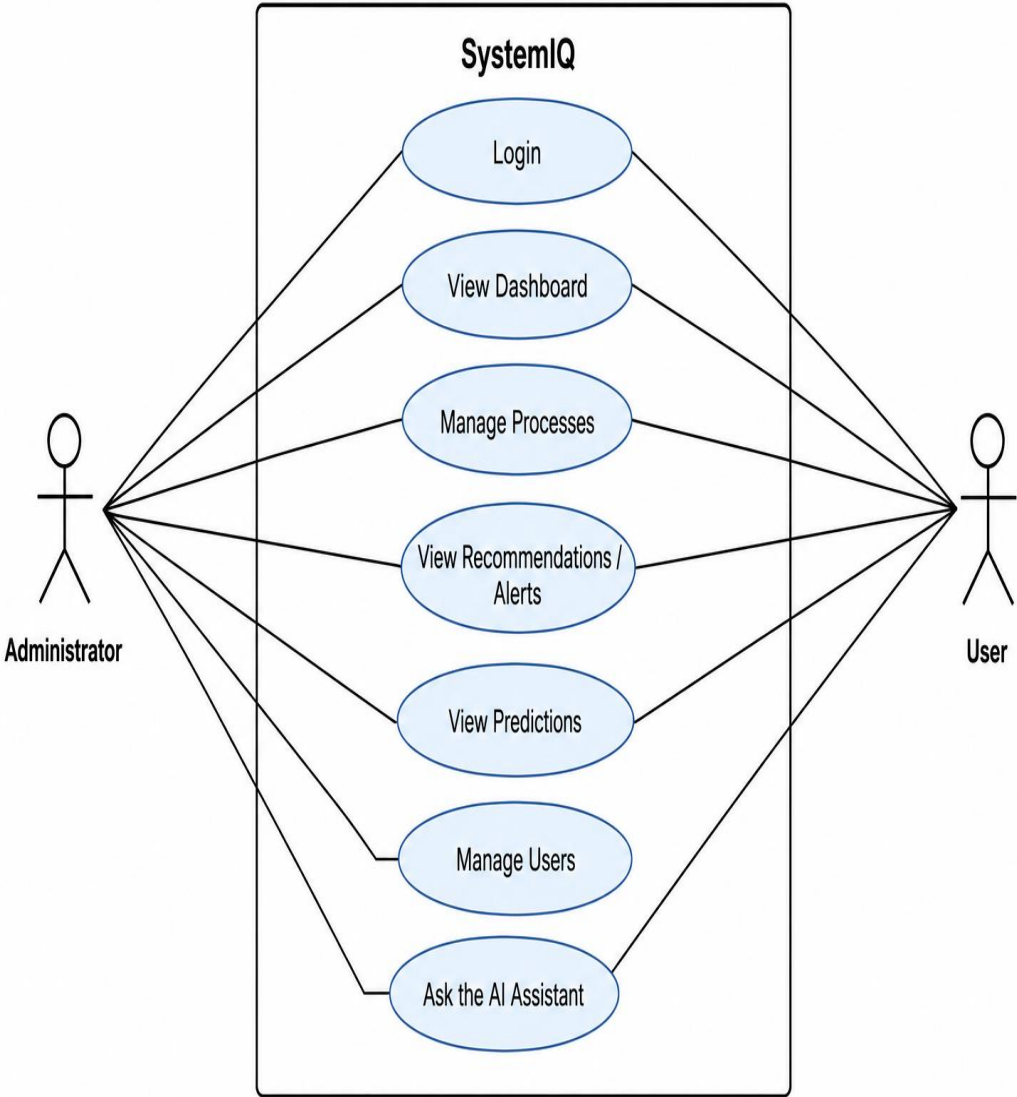


Figure 5: Use Case Diagram.

Table 4: Use case descriptions.

Use Case	Actor	Description
Login	User, Admin	Log in and receive an access token
View Dashboard	User, Admin	See the live readings and charts
Manage Processes	User, Admin	See the processes and stop one if needed
View Recommendations / Alerts	User, Admin	Read the advice and mark it as seen or solved
View Predictions	User, Admin	See the prediction charts
Manage Users	Admin only	Add, view, edit, and remove user accounts
Ask the Assistant	User, Admin	Ask a question about the system in plain words

4.6.2 Class Diagram

Figure 7 shows the main classes of the system and how they are connected.

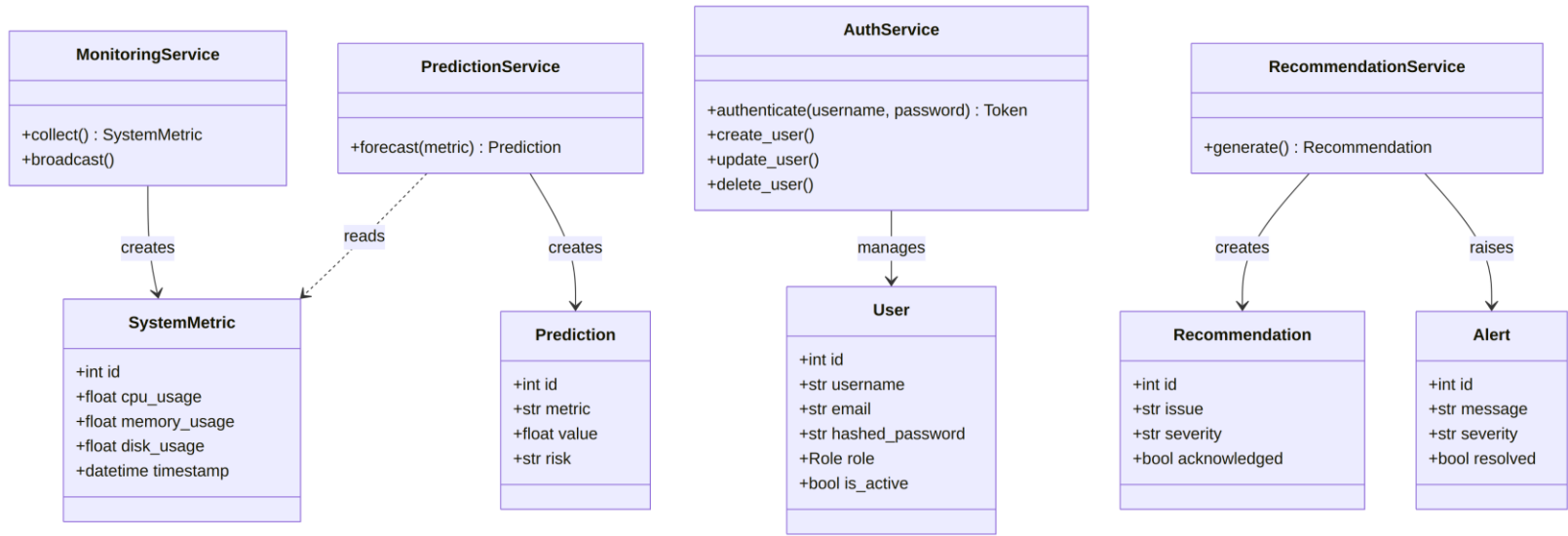


Figure 6: Class Diagram.

4.7 Time Plan (Gantt Chart)

The work was planned so that each stage followed the previous one, with some testing done along the way. Figure 8 shows the plan as a Gantt chart, and Table 6 gives the length of each stage.

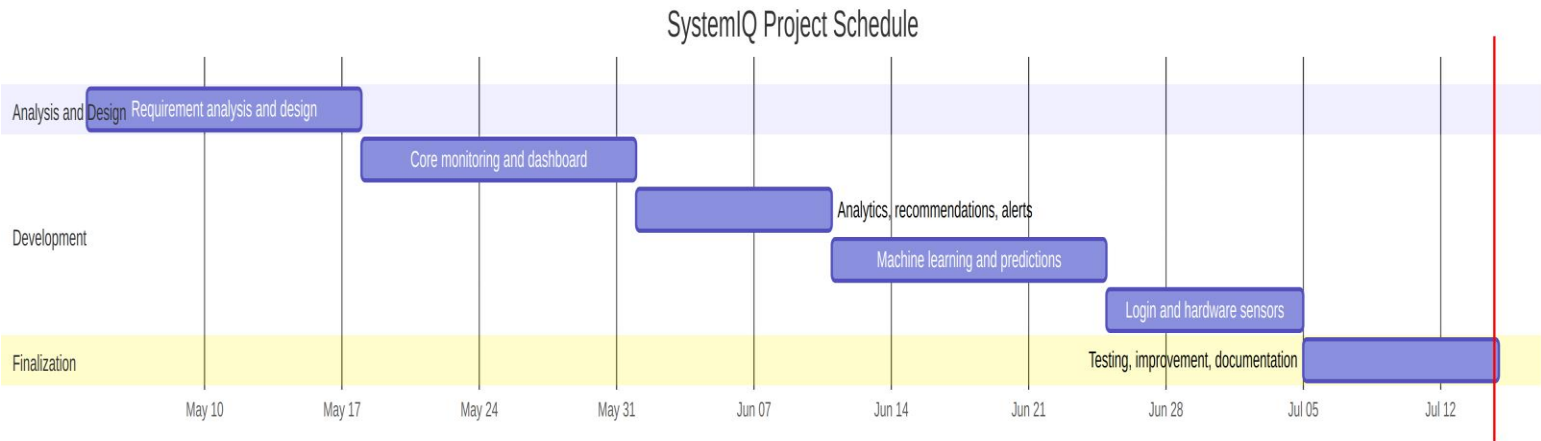


Figure 7: Gantt Chart.

Table 5: Project schedule.

Stage	Duration
Requirement analysis and design	2 weeks

Stage	Duration
Core monitoring and dashboard	2 weeks
Analytics, recommendations, and alerts	1.5 weeks
Machine learning and predictions	2 weeks
Login and hardware sensors	1.5 weeks
Testing, improvement, and documentation	1.5 weeks
Total	About 10.5 weeks

4.8 Schedule and Budget Summary

The schedule is shown in the Gantt chart and in Table 6 above.

For the budget, all of the software used in this project is free and open source, and the system runs on a computer that was already owned. Because of this, the direct cost of the project was very small. Table 7 gives the summary.

Table 6: Budget summary.

Item	Cost
Software (Python, FastAPI, React, MySQL, all free)	Rs. 0.00
Development computer (already owned)	Rs. 0.00
Hosting (runs on the local machine)	Rs. 0.00
Internet and other small costs	Rs.
Total	Rs.

CHAPTER 5: RESULTS AND DISCUSSION

5.1 Objectives

The objectives that were set for the project were:

1. To collect live readings from the machine and show them in a web dashboard in real time.
2. To store the readings in a database so that history could be studied.
3. To create recommendations and raise alerts when a limit is crossed.
4. To predict future resource usage with machine learning and show the level of risk.
5. To allow more than one user to log in with different roles and to manage user accounts.
6. To answer questions about the system in plain language and to allow safe optimization actions.

5.2 Results

By the end of the project all of the objectives had been met.

The dashboard shows the live readings and updates about every two seconds. The readings are saved in the MySQL database and can be viewed as charts for a chosen time range. Recommendations and alerts are created on their own when a resource crosses its limit. The machine-learning models predict CPU and memory usage and disk growth, show the risk level, and are retrained by the system once a day. Logging in works with different roles, and the administrator can add, view, edit, and remove users. The hardware page shows the sensor values, including the battery charge and battery health, together with temperature predictions.

Screenshots of these results are given in Appendix A (Figures 9 to 13).

5.3 Limitations

- While working on the project a few limitations were noticed:
- The prediction needs enough history to work well. When the system has only just started collecting data, the predictions are not very accurate, but they improve as more data is collected.
- Some hardware sensors are not available on every machine. On a virtual machine, for example, many sensors are missing, so the system falls back to whatever it can read.
- The fan speed (RPM) could not be read on the laptop used for development. The Linux monitoring subsystem on this machine does not expose a fan-speed sensor, so the system simply leaves the fan value empty instead of showing a wrong number. On hardware that does expose a fan sensor, this value would appear normally.
- The system watches only one machine at a time. It does not yet monitor several machines together.

5.4 Future Development

There are several ways the system could be improved in the future:

- Allow it to monitor many machines from one central dashboard.
- Add a stronger natural-language assistant.
- Make the prediction aware of daily and weekly patterns.
- Add detection of unusual behaviour, not only fixed limits.
- Provide a version that works well on mobile phones.

CHAPTER 6: CONCLUSIONS

This project demonstrates that real-time system monitoring, historical data analysis, and predictive intelligence can be successfully integrated into a lightweight and efficient platform that can be hosted and managed by an individual user. Unlike traditional monitoring tools that mainly focus on displaying current hardware and system statistics, SystemIQ provides a more intelligent approach by allowing users to analyze performance trends, receive understandable recommendations, and identify potential issues before they develop into serious problems.

The implementation of a three-layer architecture has improved the overall organization, maintainability, and scalability of the system by separating the presentation, application logic, and data management responsibilities. This structured approach makes future enhancements easier to implement, including the integration of additional monitoring capabilities, advanced machine learning models, and improved optimization techniques.

All major objectives defined at the beginning of the project have been successfully achieved. SystemIQ provides a strong foundation for intelligent system management by combining monitoring, analysis, and predictive features within a single lightweight solution. The proposed future improvements will further enhance the system's accuracy, usability, and ability to provide more advanced proactive system management capabilities.

FUTURE WORK AND RECOMMENDATIONS

FUTURE WORK AND RECOMMENDATIONS

Although SystemIQ has successfully achieved its main objectives, several improvements can be considered to increase its functionality, security, and practical usability. One of the most valuable future enhancements would be the implementation of multi-device monitoring, allowing users to monitor and manage multiple computers from a single dashboard. This feature would expand the system's usefulness beyond individual users and make it more suitable for small organizations or larger technical environments.

Another important improvement would be the integration of advanced anomaly detection techniques and a more intelligent AI assistant. By analyzing system behaviour patterns, the platform could automatically identify unusual activities, detect early signs of hardware or performance problems, and provide more accurate recommendations to users. Further improvements to predictive models could also enhance the system's ability to forecast potential failures and optimize system performance.

From a security perspective, additional measures are recommended before deploying SystemIQ in a real-world environment. The application should be hosted using a secure HTTPS connection to protect communication between users and the system. Sensitive security settings, including authentication token secret keys and administrator credentials, should be replaced with strong, unique values instead of default configurations. Regular security updates, access control improvements, and proper data protection practices should also be considered to ensure the reliability and safety of the platform.

These future enhancements would transform SystemIQ from a lightweight personal monitoring solution into a more powerful intelligent system management platform capable of supporting advanced monitoring, prediction, and automated optimization features.

REFERENCES

The following sources were used during the project. They should be written in the citation style required by the institute.

1. FastAPI documentation. <https://fastapi.tiangolo.com/>
2. React documentation. <https://react.dev/>
3. scikit-learn documentation. <https://scikit-learn.org/>
4. SQLAlchemy documentation. <https://www.sqlalchemy.org/>
5. psutil documentation. <https://psutil.readthedocs.io/>
6. MySQL documentation. <https://dev.mysql.com/doc/>

APPENDIX A: ADDITIONAL SCREENSHOTS

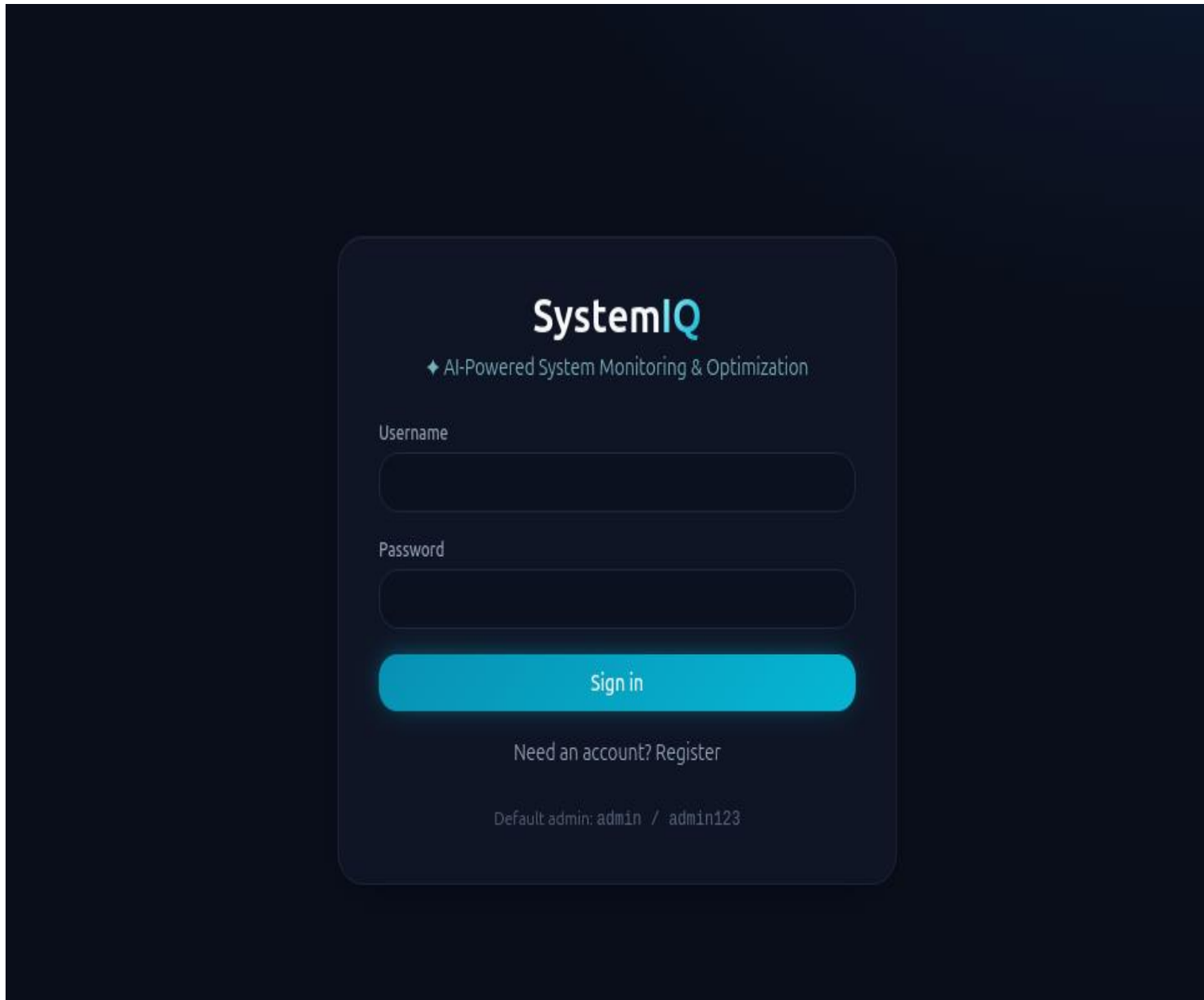


Figure 8: Login page



Figure 9: Real-time monitoring dashboard

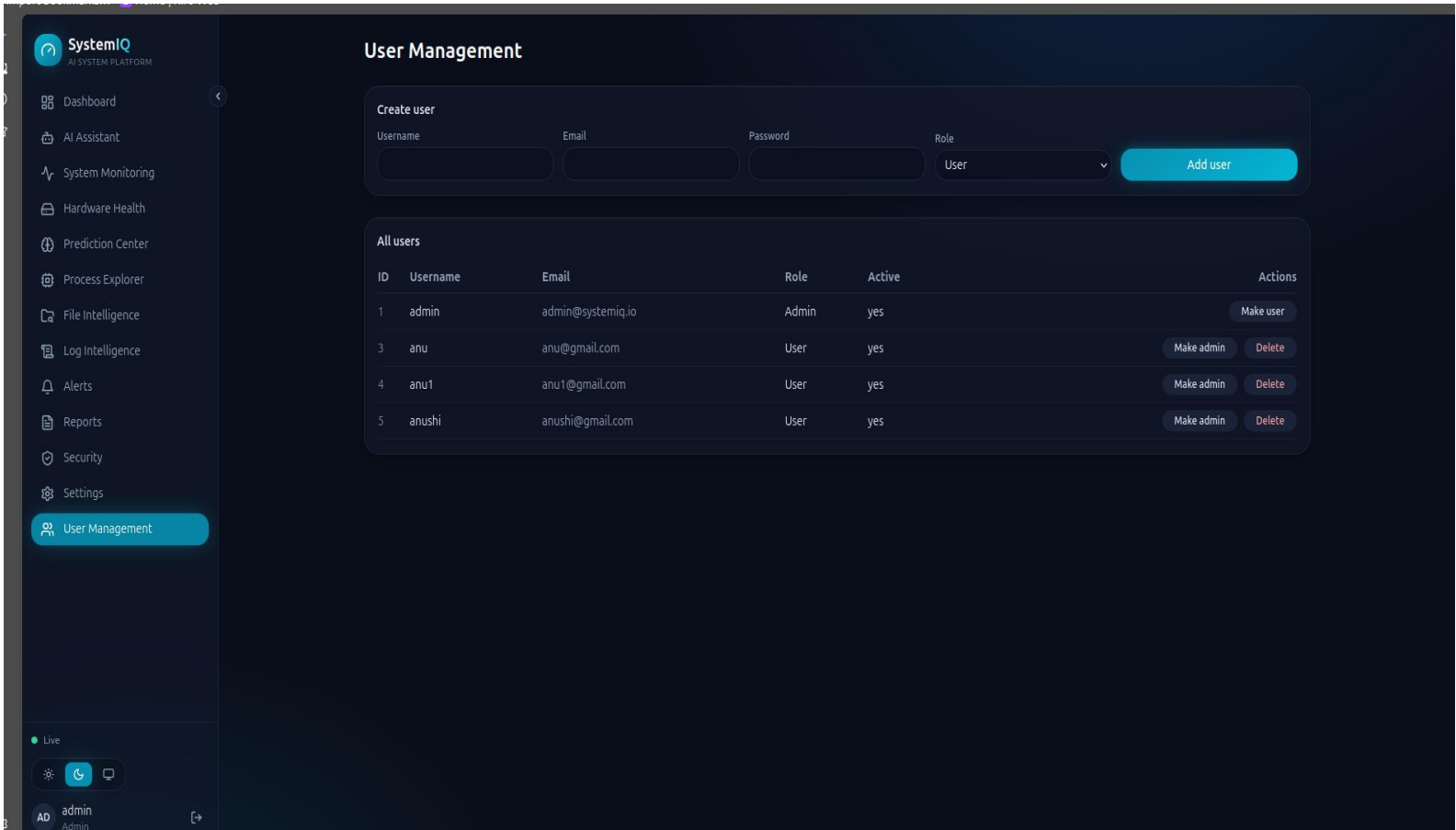


Figure 10: User management (add, view, edit, remove)



Figure 11: Predictions page



Figure 12: Hardware page

APPENDIX B: SPECIAL ALGORITHMS

This appendix contains a few important parts of the code.

Database connection

The database address is read from a setting, so the same code can work with SQLite, PostgreSQL, or MySQL without any change. For this project MySQL was used, with a connection of the form:

```
mysql+pymysql://<user>:<password>@localhost:3306/systemiq
```

```
from typing import Generator

from sqlalchemy import create_engine
from sqlalchemy.orm import DeclarativeBase, Session, sessionmaker

from app.core.config import settings

# SQLite needs a special flag to be usable across threads (FastAPI + background tasks).
_connect_args = {"check_same_thread": False} if settings.is_sqlite else {}

engine = create_engine(
    settings.DATABASE_URL,
    connect_args=_connect_args,
    pool_pre_ping=True,
    future=True,
)

SessionLocal = sessionmaker(
    bind=engine,
    autoflush=False,
    autocommit=False,
    expire_on_commit=False,
    class_=Session,
)

class Base(DeclarativeBase):
    """Declarative base shared by all ORM models."""

def get_db() -> Generator[Session, None, None]:
    """FastAPI dependency that yields a scoped database session."""
    db = SessionLocal()
```

```

try:
yield db
finally:
db.close()

def init_db() -> None:
    """Create all tables. Used for dev/SQLite; production should use Alembic."""
    # Import models so they are registered on the metadata before create_all.
    from app import models # noqa: F401 (side-effect import)

    Base.metadata.create_all(bind=engine)

```

User management with roles

The user accounts are handled by a set of endpoints that add a user, list the users, edit a user, and remove a user. All of these are protected so that only an administrator can use them.

```

"""Authentication & user-management routes."""
from __future__ import annotations

from fastapi import APIRouter, Depends, HTTPException, Response, status
from fastapi.security import OAuth2PasswordRequestForm

from app.api.deps import AdminUser, CurrentUser, DbSession
from app.core.security import create_access_token
from app.models.user import User
from app.schemas.auth import (
    LoginRequest,
    Token,
    UserCreate,
    UserRead,
    UserUpdate,
)
from app.services.auth_service import AuthService

router = APIRouter(prefix="/auth", tags=["auth"])
users_router = APIRouter(prefix="/users", tags=["users"])

def _issue_token(user: User) -> Token:
    access_token = create_access_token(subject=user.id, role=user.role.value)
    return Token(access_token=access_token, user=UserRead.model_validate(user))

```

```

@router.post("/register", response_model=UserRead, status_code=status.HTTP_201_CREATED)
def register(data: UserCreate, db: DbSession) -> UserRead:
    """Public self-registration (always creates a standard user)."""
    from app.models.enums import UserRole

    service = AuthService(db)
    # Self-registration can never escalate privileges to admin.
    data.role = UserRole.USER
    try:
        user = service.create_user(data)
    except ValueError as exc:
        raise HTTPException(status_code=status.HTTP_409_CONFLICT, detail=str(exc))
    return UserRead.model_validate(user)

@router.post("/login", response_model=Token)
def login(
    db: DbSession,
    form_data: OAuth2PasswordRequestForm = Depends(),
) -> Token:
    """OAuth2-compatible login (username + password form)."""
    user = AuthService(db).authenticate(form_data.username, form_data.password)
    if user is None:
        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED,
            detail="Incorrect username or password",
            headers={"WWW-Authenticate": "Bearer"},
        )
    return _issue_token(user)

@router.post("/login/json", response_model=Token)
def login_json(data: LoginRequest, db: DbSession) -> Token:
    """JSON login convenience endpoint for the SPA frontend."""
    user = AuthService(db).authenticate(data.username, data.password)
    if user is None:
        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED,
            detail="Incorrect username or password",
        )
    return _issue_token(user)

@router.get("/me", response_model=UserRead)
def read_me(current_user: CurrentUser) -> UserRead:

```

```

return UserRead.model_validate(current_user)

# ----- Admin user management -----
@users_router.get("", response_model=list[UserRead])
def list_users(_: AdminUser, db: DbSession) -> list[UserRead]:
    return [UserRead.model_validate(u) for u in AuthService(db).list_users()]

@users_router.post("", response_model=UserRead, status_code=status.HTTP_201_CREATED)
def create_user(data: UserCreate, _: AdminUser, db: DbSession) -> UserRead:
    try:
        user = AuthService(db).create_user(data)
    except ValueError as exc:
        raise HTTPException(status_code=status.HTTP_409_CONFLICT, detail=str(exc))
    return UserRead.model_validate(user)

@users_router.patch("/{user_id}", response_model=UserRead)
def update_user(
    user_id: int, data: UserUpdate, _: AdminUser, db: DbSession
) -> UserRead:
    service = AuthService(db)
    user = service.get_by_id(user_id)
    if user is None:
        raise HTTPException(status_code=404, detail="User not found")
    try:
        user = service.update_user(user, data)
    except ValueError as exc:
        raise HTTPException(status_code=status.HTTP_409_CONFLICT, detail=str(exc))
    return UserRead.model_validate(user)

@users_router.delete("/{user_id}", status_code=status.HTTP_204_NO_CONTENT)
def delete_user(user_id: int, admin: AdminUser, db: DbSession) -> Response:
    service = AuthService(db)
    user = service.get_by_id(user_id)
    if user is None:
        raise HTTPException(status_code=404, detail="User not found")
    if user.id == admin.id:
        raise HTTPException(status_code=400, detail="You cannot delete your own account")
    service.delete_user(user)
    return Response(status_code=status.HTTP_204_NO_CONTENT)

```

Short-term prediction

A Random Forest model is trained using the most recent readings as its input, and it predicts one step ahead at a time. When there is not enough data yet, the system simply repeats the last value as a safe fallback.

```
class ShortHorizonForecaster:
    """Lag-feature supervised forecaster for short-term metric prediction."""

    def __init__(self, lags: int = 5, model_name: str = "random_forest") -> None:
        self.lags = lags
        self.model_name = model_name
        self.model: RandomForestRegressor | LinearRegression | None = None
        self._confidence = 0.0

    def _make_model(self) -> RandomForestRegressor | LinearRegression:
        """Instantiate a fresh estimator according to `model_name`."""
        if self.model_name == "linear":
            return LinearRegression()
        return RandomForestRegressor(
            n_estimators=120, max_depth=8, random_state=42, n_jobs=-1
        )

    def _build_supervised(self, values: np.ndarray) -> tuple[np.ndarray, np.ndarray]:
        """Create (X, y) lag matrices from a 1-D series."""
        x_rows, y_rows = [], []
        for i in range(self.lags, len(values)):
            x_rows.append(values[i - self.lags : i])
            y_rows.append(values[i])
        return np.asarray(x_rows), np.asarray(y_rows)

    def fit(self, values: Sequence[float]) -> bool:
        """Train the model. Returns False if there is insufficient data."""
        arr = np.asarray(values, dtype=float)
        if len(arr) < self.lags + 5:
            self.model = None
            return False

        x, y = self._build_supervised(arr)
        model = self._make_model()
        model.fit(x, y)

        # In-sample R^2 as a rough confidence proxy (clamped to [0, 1]).
        preds = model.predict(x)
        self._confidence = float(max(0.0, min(1.0, r2_score(y, preds))))
        self.model = model
```

```

return True

def evaluate(
    self, values: Sequence[float], test_fraction: float = 0.2
) -> EvaluationResult | None:
    """Evaluate accuracy with a **chronological** hold-out split.

    Time-series data must not be shuffled, so we train on the earliest
    ``(1 - test_fraction)`` of the windows and test on the most recent
    ``test_fraction``. Each test prediction uses the actual preceding
    observations as features (one-step-ahead evaluation), which is the
    standard, honest way to score a short-horizon forecaster.

    Returns None if there is not enough data to form a meaningful split.
    """
    arr = np.asarray(values, dtype=float)
    if len(arr) < self.lags + 10:
        return None

    x, y = self._build_supervised(arr)
    n = len(x)
    split = int(n * (1.0 - test_fraction))
    if split < 1 or split >= n:
        return None

    x_train, y_train = x[:split], y[:split]
    x_test, y_test = x[split:], y[split:]

    model = self._make_model()
    model.fit(x_train, y_train)
    preds = model.predict(x_test)

    mae = float(mean_absolute_error(y_test, preds))
    rmse = float(np.sqrt(mean_squared_error(y_test, preds)))
    r2 = float(r2_score(y_test, preds)) if len(np.unique(y_test)) > 1 else 0.0

    # MAPE, guarding against division by zero.
    nonzero = np.abs(y_test) > 1e-6
    if nonzero.any():
        mape = float(
            np.mean(np.abs((y_test[nonzero] - preds[nonzero]) / y_test[nonzero])) * 100.0
        )
    else:
        mape = 0.0

```

```

return EvaluationResult(
model_name=self.model_name,
n_train=len(x_train),
n_test=len(x_test),
mae=mae,
rmse=rmse,
r2=r2,
mape=mape,
)

def forecast(self, values: Sequence[float], steps: int) -> ForecastResult:
    """Forecast `steps` future values, iteratively feeding predictions back."""
    arr = np.asarray(values, dtype=float)
    current = float(arr[-1]) if len(arr) else 0.0

    if self.model is None or len(arr) < self.lags:
        # Persistence fallback: assume the last value holds.
        return ForecastResult(
            model_name="persistence",
            confidence=0.2,
            current_value=current,
            horizon_values=[current] * steps,
        )

    window = list(arr[-self.lags :])
    forecasts: list[float] = []
    for _ in range(steps):
        x = np.asarray(window[-self.lags :]).reshape(1, -1)
        pred = float(self.model.predict(x)[0])
        pred = max(0.0, min(100.0, pred)) # clamp percentage metrics
        forecasts.append(round(pred, 2))
        window.append(pred)

    return ForecastResult(
        model_name=self.model_name,
        confidence=round(self._confidence, 3),
        current_value=current,
        horizon_values=forecasts,
    )

# ---- Persistence ----
def save(self, path: str | Path) -> None:
    import joblib

    if self.model is not None:
        Path(path).parent.mkdir(parents=True, exist_ok=True)

```

```
joblib.dump(  
{"model": self.model, "lags": self.lags, "name": self.model_name,  
"confidence": self._confidence},  
path,  
)  
  
def load(self, path: str | Path) -> bool:  
import joblib  
  
p = Path(path)  
if not p.exists():  
return False  
payload = joblib.load(p)  
self.model = payload["model"]  
self.lags = payload["lags"]  
self.model_name = payload["name"]  
self._confidence = payload.get("confidence", 0.0)  
return True
```